Stage 1: Background and Related Work

Scalable Graph Partitioning for Communication-Efficient Distributed Optimization

Team Members: Name 1, Name 2, Name 3

CS240: Algorithm Design and Analysis, Spring 2026

1 Course-Level Algorithmic Background

Before introducing the application, it is useful to recall several basic ideas from algorithm design and analysis. Many modern data problems can be made precise by first choosing a mathematical representation. In this project, the representation is a graph. A graph abstracts a system into vertices and edges: vertices are objects or computational units, while edges represent relationships, dependencies, or communication links. If edges have weights, the weights usually represent cost, similarity, capacity, or strength of interaction.

Graph problems are central in algorithms because they connect modeling, optimization, and complexity. Shortest paths, minimum spanning trees, matchings, flows, cuts, and graph partitioning are all examples where a concrete application becomes an algorithmic problem after the graph model is chosen. Among these, cut problems are especially relevant here. A cut separates a graph into different vertex sets and measures the edges crossing between them. Minimum-cut algorithms can solve some cut problems exactly, but adding extra requirements such as balance makes the problem substantially harder.

This project therefore starts from a familiar course idea—representing a problem as a graph—and then studies a harder version of a cut problem: balanced graph partitioning. The important algorithmic themes are:

- **objective functions**: minimizing cross-partition edge weight;
- **constraints**: keeping partitions balanced so that no worker receives too much work;
- **heuristics**: using practical methods when exact optimization is too expensive;
- **complexity and scalability**: studying how runtime changes as the graph becomes larger.

With this background, the project can be read as a step-by-step application of course material. Stage 1 builds the graph model and explains the related work. Stage 2 chooses and analyzes a representative algorithm. Later stages implement the algorithm, test it, and discuss possible extensions.

2 Application Background

This project studies how classical graph partitioning algorithms can be used in communication-efficient distributed optimization. In modern machine learning, scientific computing, and large-scale data processing, a problem is often too large to be solved on a single machine. A common solution is to distribute the work across several workers or agents. Each worker owns part of the data, part of the variables, or part of the computation, and the workers exchange messages to coordinate their local updates.

Communication is often one of the main bottlenecks in such systems. Local floating-point computation can be cheap compared with synchronization, cross-machine data movement, and repeated message passing. If two strongly dependent tasks are assigned to different workers, their interaction becomes a cross-worker communication event. If many such edges cross worker boundaries, the distributed algorithm may spend substantial time moving information rather than computing.

Graph partitioning provides a natural algorithmic abstraction for this issue. We model the computation as a graph $G = (V, E, w)$. Vertices represent data blocks, variable blocks, computational tasks, or agents. Edges represent dependencies, message-passing requirements, or communication links. Edge weights encode the strength or cost of communication. A partition of V into k balanced parts corresponds to assigning tasks to k workers. The desired partition should keep the workload balanced while minimizing the total weight of edges crossing between parts.

3 Algorithmic Formulation

The main computational problem is balanced k-way graph partitioning. Given an undirected weighted graph $G = (V, E, w)$ and an integer k, find disjoint sets $V_1, \dots, V_k$ such that

$$V = V_1 \cup \cdots \cup V_k, \qquad V_i \cap V_j = \emptyset \text{ for } i \neq j.$$

The cut objective is

$$\operatorname{cut}(V_1, \dots, V_k) = \sum_{(u,v)\in E,\ u\in V_i,\ v\in V_j,\ i\neq j} w_{uv}.$$

To prevent one worker from receiving almost all vertices, we also require a balance condition such as

$$|V_i| \leq (1+\epsilon)\frac{|V|}{k}, \qquad i = 1, \dots, k.$$

The optimization goal is to minimize the cut subject to this balance constraint.

This formulation connects directly to classical algorithms. Minimum cut and maximum flow solve some exact cut problems, but balanced graph partitioning is harder because the balance constraint prevents trivial solutions. Spectral methods use eigenvectors of the graph Laplacian to find low-conductance separations. Local-search methods such as Kernighan–Lin iteratively improve a partition by swapping vertices. Multilevel methods such as METIS improve scalability by coarsening the graph, partitioning a smaller graph, and refining the result on the original graph.

4 Representative Literature

Kernighan and Lin. Kernighan and Lin proposed one of the classical heuristic procedures for graph partitioning. Their method starts from an initial bisection and repeatedly selects vertex

swaps that reduce the cut. Although the original problem is graph bisection rather than general k-way partitioning, the core local-search idea remains highly influential. Our project reproduces this refinement idea and adapts it to a practical k-way setting by applying pairwise Kernighan–Lin bisection steps between parts.

Spectral graph partitioning. Spectral partitioning uses the graph Laplacian $L = D - A$, where D is the degree matrix and A is the adjacency matrix. The Fiedler vector, the eigenvector corresponding to the second-smallest eigenvalue of L, provides a relaxed solution to a graph cut problem. Sorting vertices by the Fiedler vector and splitting them gives a natural bisection. Recursive bisection then produces multiple parts. This method is important because it reveals a deep connection between graph cuts, linear algebra, and continuous relaxations.

Multilevel partitioning. Karypis and Kumar's METIS-style multilevel framework improved practical graph partitioning quality and speed. The graph is first coarsened into a sequence of smaller graphs. A partition is computed on the coarsest graph, then projected back and refined. We do not implement the full multilevel pipeline in the core project, but METIS is a key reference point because it shows how local refinement can be made scalable for large irregular graphs.

Distributed optimization over networks. Distributed subgradient, consensus, and decentralized gradient methods show how optimization algorithms behave when computation is spread across agents connected by a communication topology. In these methods, the graph controls which agents exchange information. Our project uses this setting as the modern application domain: graph partitioning is not only a graph-processing task, but also a tool for reducing cross-worker communication in distributed algorithms.

5 Why This Topic Is Algorithmically Interesting

This topic is algorithmically interesting for four reasons. First, balanced graph partitioning combines an objective function with a hard feasibility constraint. A partition with tiny cut but terrible balance is not useful for distributed computation, while a perfectly balanced partition with many cross-edges may communicate too much. The algorithm must trade off both goals.

Second, the problem illustrates the difference between exact algorithms, relaxations, and heuristics. The project compares a greedy baseline, a spectral relaxation, and a local-search refinement method. These methods represent different algorithm design strategies taught in an algorithms course.

Third, the application is modern. Graph partitioning appears in distributed machine learning, graph neural network training, sparse matrix computations, and multi-agent optimization. A classical algorithmic tool therefore helps solve a current systems problem.

Finally, the project is experimentally measurable. We can evaluate cut weight, normalized cut, balance ratio, runtime, scalability, and cross-partition communication in a simple consensus simulation. These metrics connect the formal objective to observable system behavior.

6 Project Positioning

Our selected reference topic is **Topic 2: Community Detection / Graph Partitioning**. Instead of focusing on social communities, we focus on graph partitioning for distributed optimization. This choice still fits the reference topic because the central algorithmic problem is graph partitioning, but it connects the problem to a more modern data and machine learning application.

The main method selected for reproduction is the Kernighan–Lin graph partitioning heuristic. Spectral bisection and greedy balanced partitioning are implemented as baselines. The expected outcome of the project is to demonstrate that classical graph algorithms can reduce cross-worker communication while maintaining reasonable load balance.

References

[1] B. W. Kernighan and S. Lin. An efficient heuristic procedure for partitioning graphs. *Bell System Technical Journal*, 1970.

[2] G. Karypis and V. Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 1998.

[3] A. Nedic and A. Ozdaglar. Distributed subgradient methods for multi-agent optimization. *IEEE Transactions on Automatic Control*, 2009.